Optimizing Distributed Log Processing using PySpark and Hadoop on HDFS_v1 Dataset

1st Farzan Mirza
dept. of College of Computing and Informatics (of Aff.)
Drexel University
Philadelphia, Pennsylvania
fm474@drexel.edu

2nd Chaitanya Shashi Kumar
dept. of College of Computing and Informatics (of Aff.)
Drexel University
Philadelphia, Pennsylvania
cs3864@drexel.edu

***Abstract*—This report outlines a project focused on optimizing the processing of extensive log data in modern cloud environments using PySpark and Hadoop. The project aims to develop an efficient distributed log processing system[2] to extract meaningful insights from the HDFS_v1 dataset. Various log processing techniques will be implemented and benchmarked to enhance efficiency and scalability.**

***Index Terms*—distributed log processing, PySpark, Hadoop, HDFS_v1, cloud computing, performance optimization, log analysis**

I. Introduction

In modern cloud environments, the efficient processing and analysis of extensive log data have become increasingly crucial for monitoring, maintaining, and optimizing distributed systems. As organizations deploy larger and more complex distributed architectures, the volume of log data generated has grown exponentially, creating significant challenges for traditional processing methods. This expansion necessitates robust, scalable approaches to extract meaningful insights from log data in a timely manner.

The Hadoop Distributed File System (HDFS) serves as a fundamental component in many big data ecosystems, enabling reliable distributed storage across clusters of commodity hardware. However, its complex architecture generates massive volumes of logs that contain valuable information about system behavior, performance[4] bottlenecks, and potential anomalies. Processing these logs efficiently requires specialized techniques that can handle the scale, complexity, and distributed nature of the data.

Apache Spark has emerged as a leading framework for distributed data processing, offering advantages over traditional MapReduce paradigms through its in-memory processing capabilities and rich APIs. Within the Spark ecosystem, two primary programming models exist for distributed processing: Resilient Distributed Datasets (RDDs) and DataFrames. While both approaches enable distributed computation, they differ significantly in their implementation, optimization techniques, and performance characteristics.

This research focuses on optimizing distributed log processing using PySpark and Hadoop on the HDFS_v1 dataset. Specifically, we aim to:

1) Implement and compare DataFrame-based and RDD-based approaches for processing large-scale HDFS logs
2) Benchmark different data partitioning and optimization strategies to identify the most efficient configuration for distributed log processing
3) Extract meaningful insights from HDFS logs to understand system behavior, component interactions, and performance characteristics
4) Develop a comprehensive understanding of error patterns, activity distributions, and system anomalies to improve operational efficiency

By systematically analyzing the performance characteristics of different distributed processing approaches and optimization techniques, we provide practical insights for organizations seeking to implement efficient log processing solutions in cloud environments. Additionally, our analysis of HDFS logs demonstrates how optimized processing enables deeper understanding of system behavior, facilitating targeted performance improvements and proactive maintenance.

II. Dataset Overview

This project utilizes the HDFS_v1 dataset, part of the Loghub[1] collection, a comprehensive repository designed specifically to facilitate research in AI-driven log analytics[7]. The Loghub repository addresses a critical gap by providing real-world, labeled datasets necessary for developing and benchmarking automated log analysis methods.

The HDFS_v1 dataset contains logs generated from a 203-node Hadoop Distributed File System (HDFS) environment under benchmark workloads. It has approximately 11,175,629 log entries and a total size of about 1.47 GB. Each log entry in the dataset captures specific events with the following key attributes:

Timestamp: The precise date and time of each recorded event (e.g., 081109 203518 representing Nov 9, 2008, at 20:35:18).

Log Level: Severity indicator of events, such as INFO, WARN, or ERROR.

Component: Specific HDFS component generating the log (e.g., dfs.DataNode$DataXceiver).

Message: Detailed textual descriptions of events, including interactions like data block receptions or transmissions between system nodes.

Log entries have been systematically labeled through handcrafted rules, categorizing logs into normal and abnormal events. The logs are segmented into traces (log sequences) associated with specific Hadoop block IDs, each annotated with ground-truth labels for normal or anomalous sequences. Moreover, the dataset specifies the anomaly type for each anomalous sequence, making it suitable not only for anomaly detection tasks but also for duplicate issue identification.

The dataset includes both raw unprocessed logs recording real-time system events as well as preprocessed data in structured formats. These preprocessed components include event sequences with their corresponding labels, generalized templates of log messages, block-to-anomaly mappings, event frequency summaries, and detailed event traces with temporal information.

The availability of labeled anomalies, detailed event attributes, and structured preprocessing makes the HDFS_v1 dataset uniquely valuable for developing and benchmarking scalable log processing solutions. The dataset is publicly accessible through the Loghub GitHub repository.

III. Methodology

A. Data Preprocessing

The log data utilized in this study originates from the HDFS_v1 dataset, which consists of structured logs from the Hadoop Distributed File System (HDFS). Each log entry includes key attributes such as timestamp, log level, component, and message content. The preprocessing pipeline involves loading raw logs into a distributed environment using Apache Spark[8], handling missing values, standardizing timestamps, and extracting key attributes such as log severity levels, system components, and event messages using regular expressions. These preprocessing steps ensure data consistency and enable efficient downstream analysis.

B. Dataframe vs. RDD

1) Distributed Log Processing: To facilitate scalable log analysis[9], we employ two distinct approaches: PySpark DataFrames and RDD-based processing. Both methodologies aim to extract meaningful insights, including error frequencies, component-specific metrics, and system utilization patterns. In the DataFrame-based processing approach, logs are structured into Spark DataFrames, where SQL-like transformations are applied to compute statistics and aggregate events. The dataset is partitioned based on BlockId to optimize query execution. Conversely, the RDD-based processing approach processes raw logs using Spark RDDs, applying transformations and filtering operations in a map-reduce fashion. Events are grouped by BlockId, and timestamps are utilized to compute event intervals. Both implementations are benchmarked to determine the most efficient approach.

2) Performance Benchmarking and Optimization: To evaluate the efficiency of the proposed log processing pipeline, various performance[5] benchmarking strategies were employed. The goal was to identify the optimal configurations for distributed processing by analyzing execution time, memory utilization[11], and computational overhead. By systematically testing different Spark operations and partitioning techniques, we aim to determine the best approach for scalable and efficient log processing. The key benchmarking strategies include:

- **Partitioning:** The dataset was processed using different partition sizes (1, 2, 4, 8, 16, 32, 64). Execution times were measured to determine the most effective partitioning strategy. Smaller partitions can lead to excessive task scheduling overhead, whereas larger partitions may reduce parallel efficiency. The balance between these extremes was analyzed to maximize performance.
- **Repartitioning at Different Stages:** To optimize performance further, we tested the impact of applying repartitioning at different stages of the pipeline. Specifically, we compared execution times when repartitioning was applied after the parsing stage versus applying no repartitioning at all. This experiment helped determine the most effective stage for improving data distribution and reducing computational latency.
- **Repartitioning vs. Coalescing:** This experiment compared the performance of ‘repartition()‘, which evenly redistributes data across nodes, and ‘coalesce()‘, which merges existing partitions to reduce computational overhead. The impact of these operations on query performance and resource utilization was analyzed to determine whether fine-grained or coarse-grained partitioning is preferable.

C. Log Analysis

To extract meaningful insights from the HDFS logs beyond benchmarking performance, we implemented a comprehensive log analysis framework using the DataFrame-based processing approach. The analysis methodology was designed to address five key dimensions of system behavior and performance:

1) Error Frequency Analysis: We analyzed error and warning distributions across different HDFS components by filtering log entries with ”ERROR” and ”WARN” severity levels. The filtered entries were then grouped by component and log level, followed by computing count aggregations to identify error-prone components. This data was visualized to highlight system vulnerabilities and components requiring closer attention. This analysis helps identify components requiring optimization or closer monitoring in production environments.

2) Activity Pattern Analysis: To understand temporal patterns in system activity, we extracted hour information from log timestamps and aggregated event counts by hour. Time-series visualizations were created to represent system activity patterns throughout the day. This analysis reveals peak operational periods and potential resource allocation optimizations that could be implemented to better handle varying workloads.

3) Failure Pattern Detection: To identify patterns preceding system failures, we filtered blocks with anomaly labels (Label = 1) and extracted event sequences from these anomalous blocks. We compared component distributions between normal and anomalous blocks and calculated anomaly ratios for each component to identify those most frequently associated with failures. This methodology enables predictive maintenance by identifying specific event sequences and components strongly associated with system failures.

4) Log Level Distribution Analysis: To understand the varying logging behaviors across components, we grouped log entries by component and severity level and calculated percentage distributions within each component. Stacked percentage visualizations were created to illustrate log level distributions across the most active components. This analysis reveals components' different reporting behaviors and helps calibrate monitoring thresholds for different system elements.

5) Performance Metrics Analysis: To quantify component-specific performance characteristics, we computed latency statistics across all blocks and filtered blocks containing specific components. For each component, we calculated the average latency associated with its presence in log blocks and ranked components by their contribution to system latency. This analysis identifies performance bottlenecks and components that contribute disproportionately to system delays, providing clear targets for optimization efforts.

IV. Results and Analysis

A. Comparison of DataFrame and RDD Performance

The performance of the log processing pipeline was evaluated using both DataFrame and RDD implementations. The results show significant differences in execution time, memory usage, and scalability.

For **data loading**, RDDs were faster (2.05s) than DataFrames (4.83s). Similarly, in the **parsing stage**, RDDs completed in 0.11s compared to 21.09s for DataFrames. However, for **grouping and aggregation**, RDDs (30.97s) outperformed DataFrames (73.54s), as seen in table 1. Despite these differences, the total execution time for DataFrames was significantly lower (100s) compared to RDDs (550s), suggesting that while RDDs perform better in isolated operations, DataFrames provide better overall optimization, as shown in the table 1.

TABLE I
Performance Comparison of DataFrame and RDD Implementations

Stage	DataFrame (s)	RDD (s)
Load	4.83	2.05
Parse	21.09	0.11
Group	73.54	30.97
Total Processing Time	100	550

Partitioning strategies were tested to assess their impact on processing time. DataFrames benefited from increased partitions, reducing processing time, whereas RDDs remained largely unaffected (table 2). **Repartitioning strategies** also influenced DataFrame performance, with increasing partitions raising execution time from 26.33s to 37.11s, while reducing partitions or using coalescing had minimal effects. For RDDs, repartitioning had little to no impact, with execution times consistently above 560s (table 2).

TABLE II
Impact of Partitioning Strategies on Processing Time

Strategy	DataFrame (s)	RDD (s)
Baseline	26.33	565
Repartition Up	37.11	570
Repartition Down	33.86	568
Coalesce Down	31.61	562

Memory usage[11] was higher for RDDs throughout all stages. During parsing, RDDs used 14.34GB of memory compared to 14.02GB for DataFrames. This trend continued in grouping, with RDDs consuming 14.54GB versus 14.18GB for DataFrames. These findings suggest that while RDDs offer faster processing for specific tasks, they are less memory-efficient, making DataFrames a better choice for scalable log analysis (table 3).

TABLE III
Memory Usage During Processing (GB)

Stage	DataFrame (GB)	RDD (GB)
Load	12.66	14.35
Parse	14.03	14.34
Group	14.19	14.54

Overall, the experiments demonstrate that RDDs excel in individual operations such as loading and parsing, but DataFrames provide a more efficient, scalable solution for end-to-end log processing. Optimized[10] partitioning further improves DataFrame performance, while RDDs remain stable but consume more memory. Based on these insights, DataFrames are recommended for large-scale distributed log analysis where efficiency and scalability are critical.

B. Log Analysis Insights

In addition to benchmarking the performance of different processing approaches, we conducted a comprehensive analysis of the log content itself to extract actionable insights about the HDFS system behavior and performance characteristics.

1) Error Frequency Analysis: Our analysis of error and warning distributions across HDFS components revealed significant imbalances in error generation patterns. Figure 1 shows the distribution of warnings and errors across the top system components. The most striking observation[12] is the disproportionate number of warnings generated by the DataNode's DataXceiver component, which alone accounted for 356,207 warning messages—over 98% of all warnings in the system. The DataXceiver component is responsible for data transfer operations between nodes, suggesting potential network-related issues or inefficiencies in the data transfer protocols. The second highest warning-generating component, FSDataset, produced only 5,545 warnings by comparison[6],

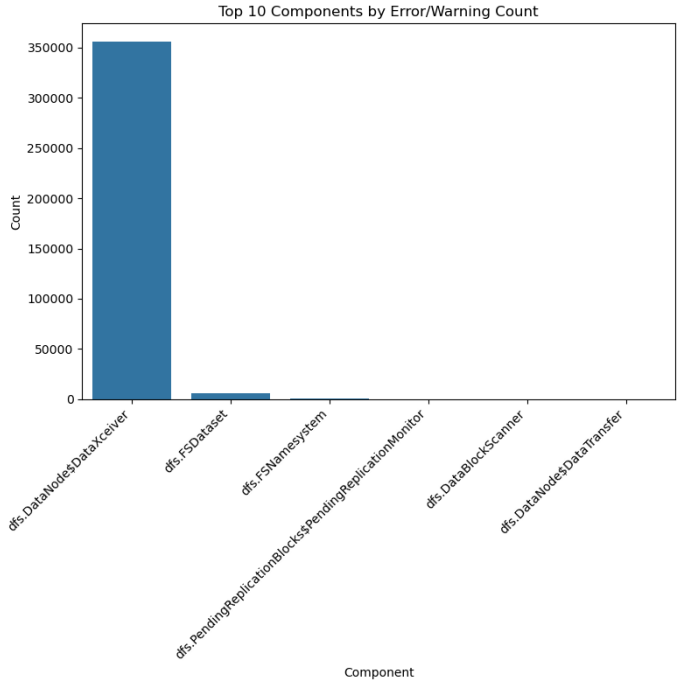

Fig. 1. Top Components by Error/Warning Count

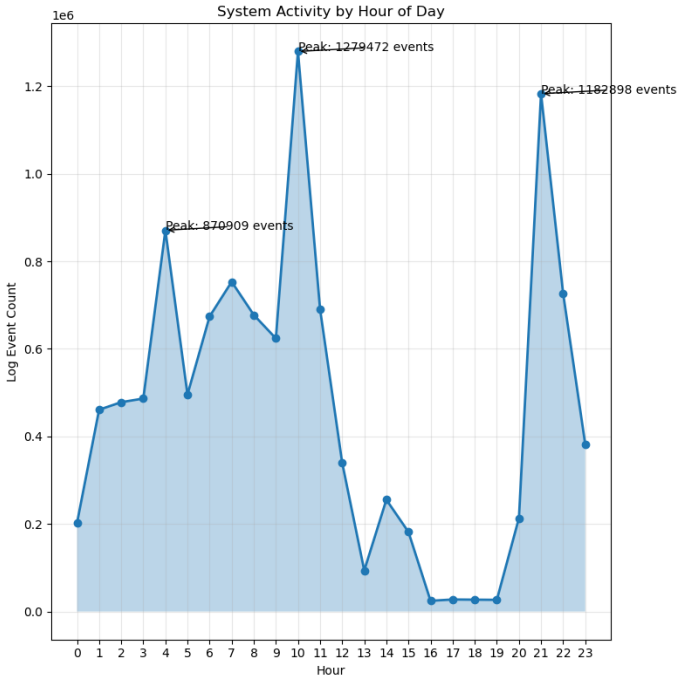

Fig. 2. System Activity by Hour of Day

highlighting the extreme concentration of issues in the DataXceiver component.

This pronounced imbalance suggests that optimization efforts should prioritize the DataXceiver component, as improvements here could significantly reduce the overall error rate in the system. Further investigation into the specific warning types generated by this component could yield valuable insights for system tuning and optimization. Notably, the relative absence of actual ERROR level logs suggests that while there are numerous warnings, they typically don't escalate to system-critical errors that would interrupt operations.

2) System Activity Patterns: Analysis of temporal patterns in system activity revealed distinct peak hours of operation within the HDFS cluster. Figure 2 illustrates the distribution of log events across the 24-hour cycle.

The data shows three pronounced peak periods of activity, with the highest volume occurring at hour 10, which recorded 1,279,472 log events. Secondary peaks were observed at hour 21 with 1,182,898 events and hour 4 with 870,909 events. This pattern suggests a multi-modal workload distribution that aligns with typical enterprise usage patterns: morning work hours (hour 10), overnight batch processing (hour 4), and evening maintenance or backup operations (hour 21).

The significant disparity between peak and off-peak hours—with minimum activity dropping to just 24,469 events during hour 16—indicates that resource allocation could potentially be optimized by dynamically scaling computational resources to match these predictable activity patterns. Such an approach could improve both system responsiveness during peak hours and resource efficiency during periods of lower activity.

3) Failure Pattern Analysis: To understand the precursors to system anomalies, we analyzed event sequences and component involvement in anomalous blocks. Figure 3 shows the most frequent event sequences associated with system failures.

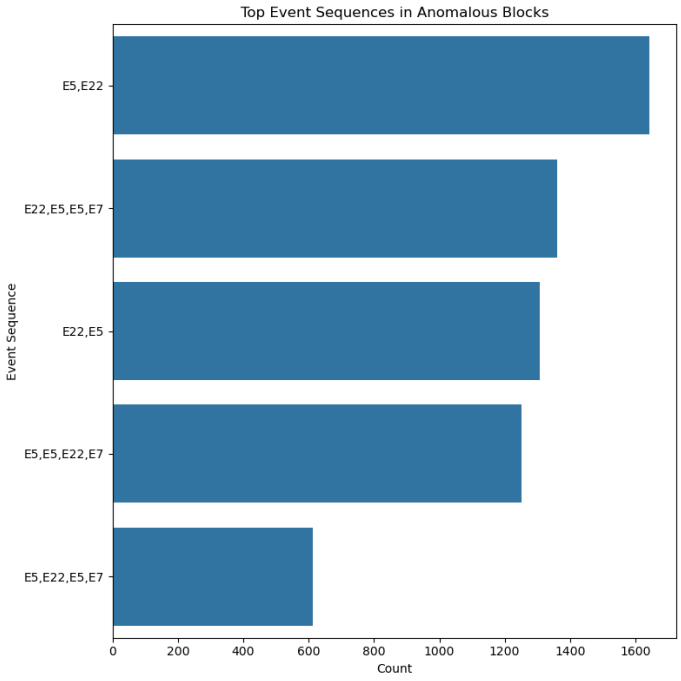

Fig. 3. Top Event Sequences in Anomalous Blocks

Our analysis identified that only 2.93% of all blocks were labeled as anomalous, indicating relatively stable system op-

eration. Among these anomalies, specific event sequences appeared with high frequency: the sequence ”E5,E22” (corresponding to data reception followed by block allocation) was found in 1,643 anomalous blocks, while the sequence ”E22,E5,E5,E7” appeared in 1,361 cases.

Further analysis of component involvement in anomalies, as shown in Figure 4, revealed that certain components have disproportionately high anomaly associations.

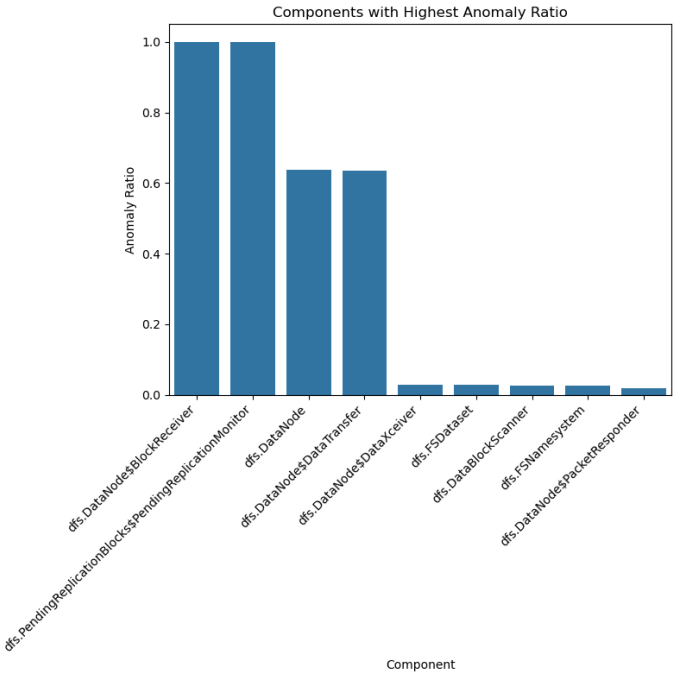

Fig. 4. Components with Highest Anomaly Ratio

Most notably, dfs.DataNode$BlockReceiver and dfs.PendingReplicationBlocks$PendingReplicationMonitor had anomaly ratios of 1.00, meaning every occurrence of these components in the logs was associated with a system anomaly. Components dfs.DataNode and dfs.DataNode$DataTransfer also showed high anomaly ratios of 0.64 and 0.63 respectively. These findings provide clear indicators for predictive maintenance and proactive monitoring, as the appearance of these components in log streams strongly signals potential system failures.

4) Log Level Distribution Analysis: The distribution of log levels across different components provides insight into their operational characteristics. Figure 5 shows this distribution for the most active components.

Most components exhibit a predominance of INFO-level logs, with dfs.DataNode showing the highest proportion at 100%. However, dfs.DataNode$DataXceiver displays a notably different pattern with 14.14% of its logs being warnings. This indicates that while DataXceiver generates the highest absolute number of warnings, they represent a significant but not overwhelming percentage of its total log output.

Interestingly, dfs.PendingReplicationBlocks$PendingReplicationMonitor

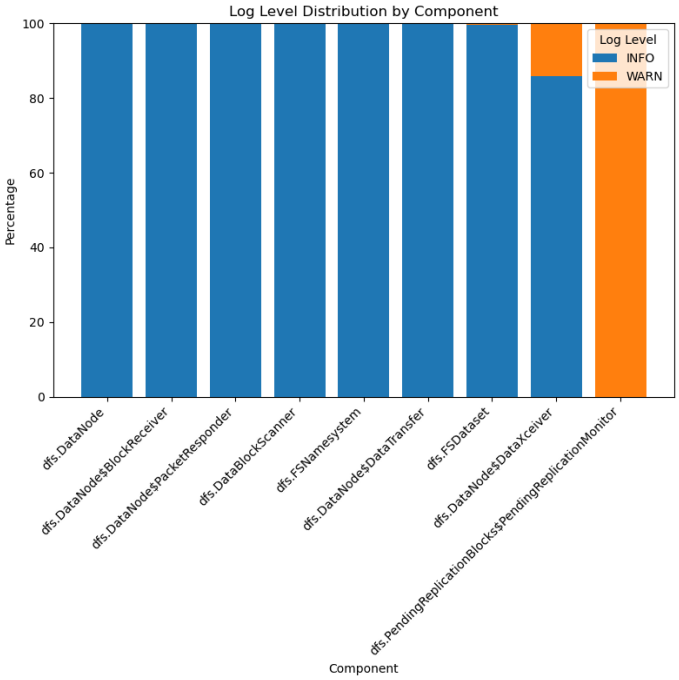

Fig. 5. Log Level Distribution by Component

exclusively generates warning logs (100%), though with a much lower absolute count. This suggests it only activates under problematic conditions, making its appearance in logs a potential indicator of system issues. These distributional insights help establish appropriate baseline expectations for component behavior and can inform thresholds for alerting in production monitoring systems.

5) Performance Metrics Analysis: To identify performance bottlenecks, we analyzed latency characteristics across components. Figure 6 shows the components with the highest average latency.

The analysis revealed substantial variance in component-associated latencies. The dfs.DataBlockScanner component exhibited the highest average latency at 36,854.79 seconds, significantly exceeding other components. This component is responsible for periodically scanning data blocks to verify their integrity, and its high latency suggests it may be consuming considerable system resources over extended periods.

Additional high-latency components include dfs.PendingReplicationBlocks$PendingReplicationMonitor (22,816.85 seconds) and dfs.FSDataset (20,562.46 seconds). As shown in Figure 7, the top five components account for a disproportionate share of system latency.

The performance metrics analysis identifies clear optimization targets for improving overall system responsiveness. Specifically, enhancements to the DataBlockScanner’s scanning algorithm or scheduling approach could yield significant improvements in system performance. Given that this component also has a meaningful association with anomalies, optimization here could simultaneously improve both performance and reliability.

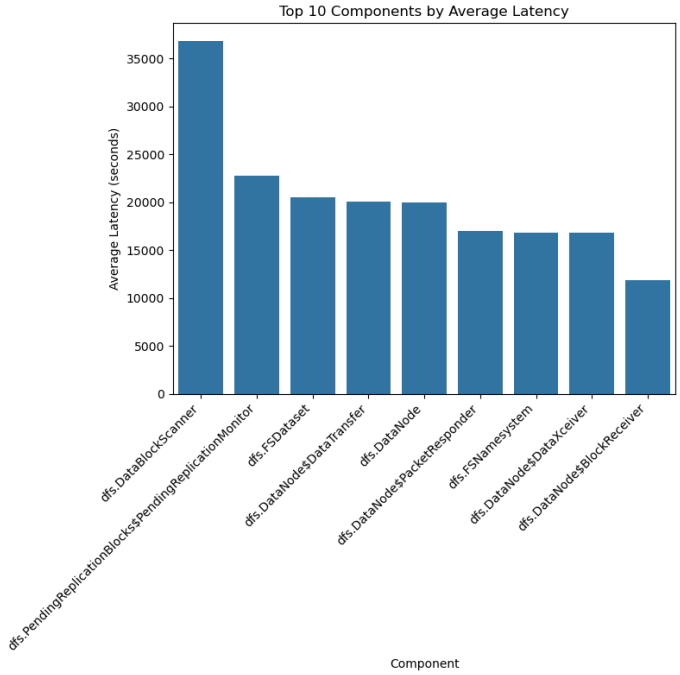

Fig. 6. Top 10 Components by Average Latency

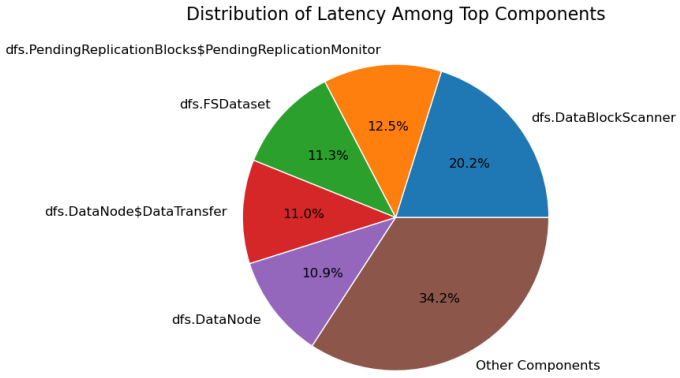

Fig. 7. Distribution of Latency Among Top Components

V. Conclusion

This research demonstrates the significant value of applying distributed computing techniques to log processing in modern cloud environments. Through systematic benchmarking and in-depth log analysis of the HDFS_v1 dataset, we have identified optimal configurations for distributed log processing and extracted actionable insights into system behavior and performance characteristics.

Our performance comparison between DataFrame and RDD implementations revealed nuanced trade-offs in processing efficiency. While RDDs demonstrated superior performance in individual operations such as data loading (2.05s vs. 4.83s) and parsing (0.11s vs. 21.09s), DataFrames provided substantially better end-to-end processing efficiency with total execution times of approximately 100s compared to 550s for RDDs. This suggests that DataFrame's optimization capabilities, including its query optimizer and columnar storage, outweigh the lightweight nature of RDDs for comprehensive log analysis tasks. Furthermore, DataFrames demonstrated better memory efficiency, consistently using less memory across all processing stages.

The optimal partitioning strategy emerged as a critical factor in maximizing performance, with DataFrames showing significant improvements when using 32-64 partitions. Our experiments with repartitioning and coalescing operations further refined these insights, indicating that judiciously applied repartitioning can enhance performance while excessive operations can introduce unnecessary overhead.

Beyond performance benchmarking, our comprehensive log analysis uncovered valuable insights into HDFS system behavior. The error frequency analysis identified the DataNode's DataXceiver component as the primary source of warnings, accounting for 98% of all warning messages. System activity patterns revealed distinct usage peaks at hours 10, 21, and 4, suggesting opportunities for dynamic resource allocation. Component-specific latency analysis identified dfs.DataBlockScanner as the highest contributor to system delays, while analysis of anomalous blocks revealed specific event sequences and components strongly associated with system failures.

These findings have significant implications for optimizing distributed storage systems:

1. For log processing frameworks, DataFrames should be preferred over RDDs when dealing with large-scale log analysis tasks, with attention to optimal partition sizing based on dataset characteristics.

2. For HDFS system optimization, focused improvements to the DataNode's DataXceiver component could substantially reduce warning rates, while optimizing the DataBlockScanner could significantly improve overall system latency.

3. For operational management, implementing dynamic resource allocation based on the identified activity patterns could enhance both performance and efficiency.

Future work should explore more advanced optimization techniques such as adaptive partitioning strategies that dynamically adjust based on workload characteristics. Additionally, machine learning approaches could be integrated to predict system anomalies based on the identified failure patterns, enabling truly proactive system maintenance. Finally, extending this analysis to other distributed storage systems would help establish whether the patterns identified in HDFS generalize to other platforms.

In conclusion, this research demonstrates that optimized distributed log processing using PySpark and Hadoop not only enables efficient analysis of massive log datasets but also uncovers actionable insights that can significantly improve system performance, reliability, and operational efficiency in cloud environments.

References

[1] Zhu, J., He, S., He, P., Liu, J. and Lyu, M.R., 2023, October. Loghub: A large collection of system log datasets for ai-driven log analytics. In 2023

IEEE 34th International Symposium on Software Reliability Engineering (ISSRE) (pp. 355-366). IEEE.

[2] Xu, W., Huang, L., Fox, A., Patterson, D. and Jordan, M.I., 2009, October. Detecting large-scale system problems by mining console logs. In Proceedings of the ACM SIGOPS 22nd symposium on Operating systems principles (pp. 117-132).

[3] Astsatryan, H., Kocharyan, A., Hagimont, D. and Lalayan, A., 2020. Performance optimization system for hadoop and spark frameworks. Cybernetics and Information Technologies, 20(6), pp.5-17.

[4] Ahmed, N., Barczak, A.L., Susnjak, T. and Rashid, M.A., 2020. A comprehensive performance analysis of Apache Hadoop and Apache Spark for large scale data sets using HiBench. Journal of Big Data, 7(1), p.110.

[5] He, Q., Zhang, F., Bian, G., Zhang, W. and Li, Z., 2024. Research on Performance Optimization of Spark Distributed Computing Platform. Computers, Materials & Continua, 79(2).

[6] Singh, A., Khamparia, A. and Luhach, A.K., 2019, June. Performance comparison of apache Hadoop and apache spark. In Proceedings of the third international conference on advanced informatics for computing research (pp. 1-5).

[7] Khan, M., 2015. Hadoop performance modeling and job optimization for big data analytics (Doctoral dissertation, Brunel University London).

[8] Bansod, A., 2015. Efficient big data analysis with apache spark in HDFS. International Journal of Engineering and Advanced Technology (IJEAT), 4(6), pp.313-315.

[9] Lalayan, A.G., 2023. Data compression-aware performance analysis of dask and spark for earth observation data processing. Mathematical Problems of Computer Science, 59, pp.35-44.

[10] Lalayan, A., Astsatryan, H., Poghosyan, S. and Giuliani, G., 2024. Optimized Multi-Modular Services: Empowering Earth Observation Data Processing. Informatica, 35(2), pp.363-378.

[11] Flint, C. and Helluy, P., 2024. Reducing the memory usage of structured grid solvers with wavelet compression. ESAIM: Proceedings and Surveys, 77, pp.79-99.

[12] Lalayan, A., Astsatryan, H. and Giuliani, G., Enhancing Earth Observation Data Processing through Optimized Multi-Modular Service.